Lex: A Programming Language for Developing Enforceable Legal Formalizations

Jonas Degelo, François Hublet, and Jakob Merane

ETH Zürich, Switzerland
{degeloj,fhublet,jmerane}@ethz.ch

Abstract. **Keywords:** Legal formalization · Programming languages.

1 Introduction

2 Logical foundations

3 Syntax

3.1 Modules

Every `Lex` file defines a module, while folders containing `Lex` files define groups of modules. The syntax of module imports is as follows:

$$\begin{aligned} \textit{import} &::= \texttt{import}\ [\texttt{formex} \mid \texttt{akomaNtoso}]\ \textit{idents} \\ \textit{idents} &::= \textit{ident} \mid \textit{idents}\,.\,\textit{ident} \end{aligned}$$

The special flags `formex` and `akomaNtoso` instruct the `Lex` compiler to load the corresponding modules as XML files containing a machine-readable version of the law, rather than `Lex` files. Two formats are supported: FORMEX (the standard EU format for machine-readable legal documents) and Akoma Ntoso (an international standard used, e.g., in Switzerland).

When the target module is a `Lex` module, the sequence $\textit{ident}_1\,.\,\ldots\,.\,\textit{ident}_n$ refers to the file $\textit{ident}_1/\ldots/\textit{ident}_n$`.lex`; if the target module is an XML module in FORMEX or Akoma Ntoso format, the file is $\textit{ident}_1/\ldots/\textit{ident}_n$`.xml`.

3.2 Labels

Labels are used to capture the structure of the law. Their syntax is as follows:

$$\begin{aligned} \textit{label\_level} &::= \texttt{law} \mid \texttt{title} \mid \texttt{chapter} \mid \texttt{section} \\ &\quad \mid \texttt{article} \mid \texttt{paragraph} \mid \texttt{point} \mid \texttt{subpoint} \\ \textit{label} &::= \textit{label\_level}[\,[\textit{int}]\,]\ \textit{string}\ [\textit{string}] \end{aligned}$$

The optional integer parameter that follows the level can be used to encode an arbitrary number of additional hierarchical levels: `chapter[1]` is one level below `chapter` but still above `section`, `chapter[`i`]` is one level below `chapter[`$i-1$`]` and one level above `chapter[`i`+1]`, etc. The second and (optional) third argument collect the number and header of the corresponding part of the law.

Example 1. Art. 2 GDPR can be introduced as

`article` "*2*" "*Material scope*"

The levels `law`, `article`, `paragraph`, `point`, and `subpoint` plain a special role in determining the *qualified name* of `Lex` rules. Rules can only defined within articles, which can themselves only defined within laws. The induced uniqueness constraints are shown in Table 1. The qualified name of a rule is of the form

law article[`(`*paragraph*`)`][`(`*point*`)`][`(`*subpoint*`)`] `#`*rule_id*

where additional level numbers placed in brackets can be introduced if intermediate, indexed levels are present. Rules introduced within the scope of a given label can be referred to using a suffix of their qualified name that omits the part determined by the scope. The set of all rules located within the scope of a given label can be referred to using the prefix of their qualified name corresponding to that scope, or by a suffix of this prefix, if applicable within the current scope.

Example 2. The set of rules defined in Art. 2, §2 of the GDPR can be referred to as "*GDPR 2(2)*". Within the GDPR, they can be referred to as just "*2(2)*", and within Art. 2, as "*(2)*". The first of such rules is "*GDPR 2(2) #1*", the second one is "*GDPR 2(2) #2*", etc. The rule IDs 1..*k* can be overwritten by defining explicit rule IDs (see the syntax of rules below).

Label level	Number unique within	Part of qualified name	Required
`law`	–	✓	✓
`title`	`law`		
`chapter`	`law`		
`section`	`law`		
`article`	`law`	✓	✓
`paragraph`	`article`	✓	
`point`	`paragraph`	✓	
`subpoint`	`point`	✓	

Table 1: Uniqueness constraints for labels

3.3 Types

`Lex` has four base types: `int` (integers), `float` (floating-point numbers), `bool` (booleans), and `str` (strings). New types can be declared as follows:

type_decl ::= `type` *ident* [[`is`|`subtype`] *ident*]

When another, already existing type name is specified after the `is` keyword, the newly declared type is created as a copy of that existing type. As a consequence,

the new type will have the same internal representation and take the same values as the existing type. However, the existing type and the new type will still be considered distinct types by `Lex`'s type system (see example below). In contrast, the `subtype` keyword support specifying a new type as a subtype of an existing type. In this case, the new type will have both the same internal representation and the capacity to be used in any context where the existing type could be used.

Example 3. Consider the following declarations:

```
type id is string
type user_id subtype id
```

Further, consider an event with signature `A(b:id, c:string)` and a variable `uid` of type `user_id`. The expression `A(uid, "foo")` is type-correct, since `user_id` is a subtype of `id`. The expression `A(uid, uid)` is not type-correct, since `user_id` is not a subtype of `string` (because `id` is not a subtype of `string`). On the other hand, `A("foo", "foo")` is type-correct since the type `id` can contain string values.

3.4 Events

The syntax for event declarations is as follows:

capabilities ::= `observable` | `suppressable` | `causable` | `internal`
| `observable causable` | `causable observable`
| `suppressable causable` | `causable suppressable`

event_decl ::= *capabilities* `event` *ident*
[`"""`*event_desc*`"""`]
$ident_{11}$ `:` $ident_{12}$
...
$ident_{k1}$ `:` $ident_{k2}$

where *event_desc* is a string that can contain special placeholders `{`$ident_{i1}$`}` for $1 \leq i \leq k$. The capabilities associated to an event are organized into the following lattice, where $a \to b$ denotes that a is strictly weaker than b.

The pairs of identifiers $ident_{i1}$ `:` $ident_{i2}$ contained in the event declaration determine an (ordered) list of event parameters ($ident_{i1}$) and their types ($ident_{i2}$). Finally, the event description *event_desc* provides the high-level semantics of the event, which should refer to the parameters $ident_{i1}, \ldots, ident_{k1}$ using the `{`$ident_{i1}$`}` syntax.

Example 4. The following code declares a new suppressable event capturing personal data processing in a system, assuming three previously defined types

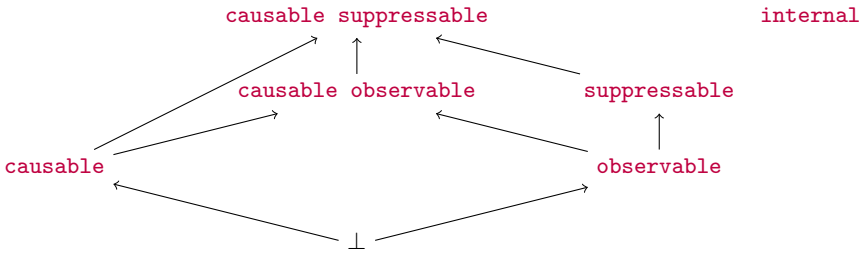

Fig. 1: Capability lattice

processing_id, processor_id, and data_id.

```
suppressable event PersonalDataProcessing
"""
processor {x} is processing personal data {z} as part
of processing operation {ep}
"""
    ep : processing_id
    x  : processor_id
    z  : data_id
```

3.5 Rules

The general syntax for rule declarations is as follows:

$$
\begin{aligned}
\mathit{rule\_verb\_action} ::= {} & \texttt{oblige} \mid \texttt{permit} \mid \texttt{prohibit} \mid \texttt{constitute} \\
\mathit{rule\_verb\_relative} ::= {} & \texttt{except} \mid \texttt{scope} \\
\mathit{rule\_constr\_verb} ::= {} & \texttt{causing} \mid \texttt{suppressing} \\
\mathit{rule\_constr} ::= {} & \mathit{rule\_constr\_verb}\ \mathit{ident}_1\ \ldots\ \mathit{ident}_m \\
\mathit{rule\_action} ::= {} & \mathit{rule\_verb\_action}\ [\mathit{patt}_2] \\
& [\mathit{ident}_{41}\texttt{:}]\ \mathit{expr}_{41} \\
& \ldots \\
& [\mathit{ident}_{4k_4}\texttt{:}]\ \mathit{expr}_{4k_4} \\
\mathit{rule\_reference} ::= {} & \texttt{\{}\ \mathit{label\_level}_1[\texttt{[}\mathit{int}_1\texttt{]}]\ \mathit{string}_1\ \ldots\ \mathit{label\_level}_k[\texttt{[}\mathit{int}_k\texttt{]}]\ \mathit{string}_k\ [\texttt{rule}\ \mathit{string}_{k+1}]\ \texttt{\}} \\
\mathit{rule\_relative} ::= {} & \mathit{rule\_verb\_relative}\ [\mathit{patt}_2] \\
& \mathit{rule\_reference}_1 \\
& \ldots \\
& \mathit{rule\_reference}_l \\
\mathit{rule\_decl} ::= {} & \texttt{rule}\ [\mathit{string}] \\
& [\texttt{fix} \\
& \quad \mathit{ident}_{11}\texttt{:}\mathit{ident}_{21} \\
& \quad \ldots \\
& \quad \mathit{ident}_{1n}\texttt{:}\mathit{ident}_{2n}] \\
& [\texttt{whenever}\ [\mathit{patt}_1] \\
& \quad [\mathit{ident}_{31}\texttt{:}]\ \mathit{expr}_{31} \\
& \quad \ldots \\
& \quad [\mathit{ident}_{3k_3}\texttt{:}]\ \mathit{expr}_{3k_3}] \\
& (\mathit{rule\_action} \mid \mathit{rule\_relative}) \\
& [[\texttt{transparently}]\ \texttt{enforceable}]\ \mathit{rule\_constr}_1\texttt{,}\ \ldots\ \texttt{,}\mathit{rule\_constr}_\ell
\end{aligned}
$$

In the following we review the syntax of expressions expr_{ij} (Section 3.5), the available verbs (Section 3.5) and the temporal patterns determined by patt_1 and patt_2 (Section 3.5).

Expressions. The syntax of expressions closely follows MFOTL syntax:

$$
\begin{aligned}
\mathit{expr\_itv} &::= \texttt{[}\mathit{time\_bound}_1\texttt{,(}\mathit{time\_bound}_2 \mid \texttt{*)]} \\
\mathit{term} &::= (\mathit{const} \mid \mathit{ident})[\texttt{:}\mathit{ident}] \\
\mathit{pred} &::= \mathit{ident}\texttt{(}\mathit{term}_1\texttt{,} \ldots \texttt{,}\mathit{term}_k\texttt{)} \\
\mathit{expr} &::= \texttt{TRUE} \mid \texttt{FALSE} \mid \mathit{pred} \mid \mathit{special\_pred} \\
&\quad \mid \texttt{NOT}\ \mathit{expr} \mid \mathit{expr}_1\ (\texttt{AND} \mid \texttt{OR} \mid \texttt{IMPLIES} \mid \texttt{EQUIV})\ \mathit{expr}_2 \\
&\quad \mid (\texttt{EXISTS} \mid \texttt{FORALL})\ \mathit{expr} \\
&\quad \mid (\texttt{ONCE} \mid \texttt{EVENTUALLY} \mid \texttt{HISTORICALLY} \mid \texttt{ALWAYS})\ [\mathit{expr\_itv}]\ \mathit{expr} \\
&\quad \mid \mathit{expr}_1\ (\texttt{SINCE} \mid \texttt{UNTIL} \mid \texttt{TRIGGER} \mid \texttt{RELEASE})\ [\mathit{expr\_itv}]\ \mathit{expr}_2 \\
\mathit{special\_pred} &::= \texttt{obliged}(\mathit{pred}) \mid \texttt{permitted}(\mathit{pred}) \mid \texttt{prohibited}(\mathit{pred}) \\
&\quad \mid \texttt{applicable}(\mathit{string}) \mid \texttt{satisfied}(\mathit{string}) \mid \texttt{violated}(\mathit{string})
\end{aligned}
$$

Verbs.

Temporal patterns. Except for `except` and `scope` rules, every `Lex` rule declaration represents a statement of the form

$$
\begin{aligned}
&\texttt{FORALL}\ \mathit{ident}_{11}, \ldots, \mathit{ident}_{1n}. \\
&\quad \texttt{ALWAYS}(\mathit{form}_1\ (\mathit{expr}_{11}\ \texttt{AND}\ \ldots\ \texttt{AND}\ \mathit{expr}_{1k}) \\
&\qquad \texttt{IMPLIES}\ \mathit{form}_2\ (\mathit{expr}_{21}\ \texttt{AND}\ \ldots\ \texttt{AND}\ \mathit{expr}_{2k}))
\end{aligned}
$$

where form_1 and form_2 are determined by patt_1 and patt_2, respectively. These options use the following grammars for time and time intervals:

$$
\begin{aligned}
\mathit{time\_unit} &::= \texttt{y} \mid \texttt{d} \mid \texttt{h} \mid \texttt{s} \mid \texttt{ms} \\
\mathit{time\_bound} &::= \mathit{int}\ \mathit{time\_unit} \\
\mathit{time\_interval} &::= \texttt{within}\ \mathit{time\_bound} \mid \texttt{before}\ \mathit{time\_bound} \mid \texttt{after}\ \mathit{time\_bound}
\end{aligned}
$$

Table 2 lists available patt_i and the corresponding form_i functions, as well as the conversion of *time_interval* expressions into intervals:

4 Compilation

4.1 To MFOTL

4.2 To LegalRuleML

4.3 To documentation

References

1. Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security (TISSEC)*, 3(1):30–50, 2000.

itv	I	Restriction
–	$[0, \infty)$	
`within` b	$[0, b]$	
`before` a	$[a, \infty)$	Only past patterns
`strictly before` a	(a, ∞)	Only past patterns
`after` a	$[a, \infty)$	Only future patterns
`strictly after` a	(a, ∞)	Only future patterns
`between` a `and` b	$[a, b]$	
`strictly between` a `and` b	(a, b)	
`between` a `and` b `excluded`	$[a, b)$	
`between` a `excluded and` b	$(a, b]$	

Description	$patt_i$	$form_i(\phi)$	Type
	–	ϕ	Present
	`eventually` [*itv*]	`EVENTUALLY`$_I$ ϕ	Future
	`eventually delaying if` ψ [*itv*]	ψ `UNTIL`$_I$ ϕ	Future
	`always in the future` [*itv*]	`ALWAYS`$_I$ ψ	Future
	`once` [*itv*]	`ONCE`$_I$ ψ	Past
	`always in the past` [*itv*]	`HISTORICALLY`$_I$ ψ	Past

Table 2: Available temporal patterns for `Lex` rules

2. Jan Chomicki. Efficient checking of temporal integrity constraints using bounded history encoding. *ACM Transactions on Database Systems (TODS)*, 20(2):149–186, 1995.
3. David Basin, Felix Klaedtke, Samuel Müller, and Eugen Zălinescu. Monitoring metric first-order temporal properties. *Journal of the ACM (JACM)*, 62(2):1–45, 2015.